Lab 2

Hardware Manipulation and Timers with ESP32

CSE/EE 474 introduction to Embedded Systems

Week -2-
SU 2026

Labs Timeline & Objective

Lab1

- Install & Configure Arduino
- Apply basic programming concepts to configure outputs
- Develop a simple program that manipulates an LED's blinking pattern.

Lab2

- Manipulate memory using pointers.
- Control hardware directly through registers.
- Implement timing exercises to manage tasks.
- Interface with LEDs, sensors, and buzzers.

Lab3

- Implement I2C communication with peripherals.
- Explore basic non-preemptive scheduling algorithms.
- Implement and manage simple Task Control Blocks (TCBs).

Lab4

- Implement preemptive scheduling algorithms.
- Explore the dual-core architecture of the ESP32 for parallel computing.
- Integrate scheduling techniques and parallel computing using FreeRTOS.

Review:

Bitwise Operators

Manipulate individual bits within a byte or a word

Pointers

Store memory addresses, granting the ability to directly manipulate memory contents.

Registers

- Small storage location within a CPU or microcontroller
- Used for temporary data storage
- Three main types:
  1. General purpose
  2. Special purpose
  3. Status registers

Bitwise Operations

```
volatile uint8_t reg = 0b00000000;

// set a bit with the OR operator
reg |= (1 << 2); // reg is now 0b00000100

// clear a bit with the AND operator and a
negated bitmask
reg &= ~(1 << 2); // reg is back to
0b00000000

// toggle a bit with the XOR operator
reg ^= (1 << 3); // reg is now 0b00001000
reg ^= (1 << 3); // reg is back to
0b00000000
```

Operator	How It Feels
& (AND)	Both Agree
|| (OR)	One Agrees
^ (XOR)	Agrees only when one doesn't
~ (NOT)	This time will pass
<< (Left Shift)	Like adding zeros to your bank account (if only!).
>> (Right Shift)	Removing the zero you added

Review:

Bitwise Operators

Manipulate individual bits within a byte or a word

Pointers

Store memory addresses, granting the ability to directly manipulate memory contents.

Registers

- Small storage location within a CPU or microcontroller
- Used for temporary data storage
- Three main types:
  1. General purpose
  2. Special purpose
  3. Status registers

Pointers

*

Pointers store the address of another variable:

```
int var1 = 10;

double var2 = 10.2;

int* iPtr = &var1; // integer pointer that
holds the address in memory of var1

double* dPtr = &var2; // holds the address
of var2


// dereference operator * to access the
contents stored at the address iPtr points
to

int newVar = *iPtr;  // so newVar is equal
to 10
```

Review:

Bitwise Operators

Manipulate individual bits within a byte or a word

Pointers

Store memory addresses, granting the ability to directly manipulate memory contents.

Registers

- Small storage location within a CPU or microcontroller
- Used for temporary data storage
- Three main types:
  1. General purpose
  2. Special purpose
  3. Status registers

Registers

Register 6.2. GPIO_OUT_REG (0x0004)

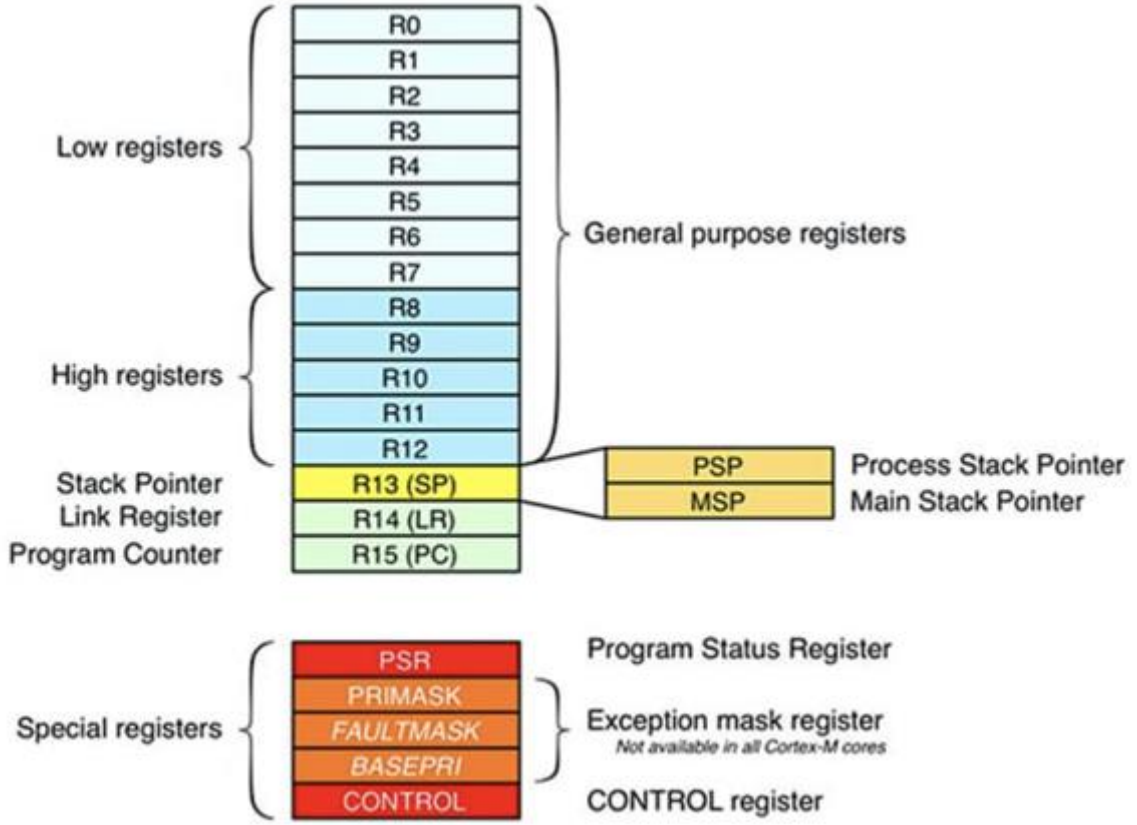
GPIO_OUT_DATA_ORIG	
31 ... 0	
0x000000	Reset

GPIO_OUT_DATA_ORIG GPIO0 ~ 21 and GPIO26 ~ 31 output values in simple GPIO output mode. The values of bit0 ~ bit21 correspond to the output values of GPIO0 ~ 21, and bit26 ~ bit31 to GPIO26 ~ 31. Bit22 ~ bit25 are invalid. (R/W)

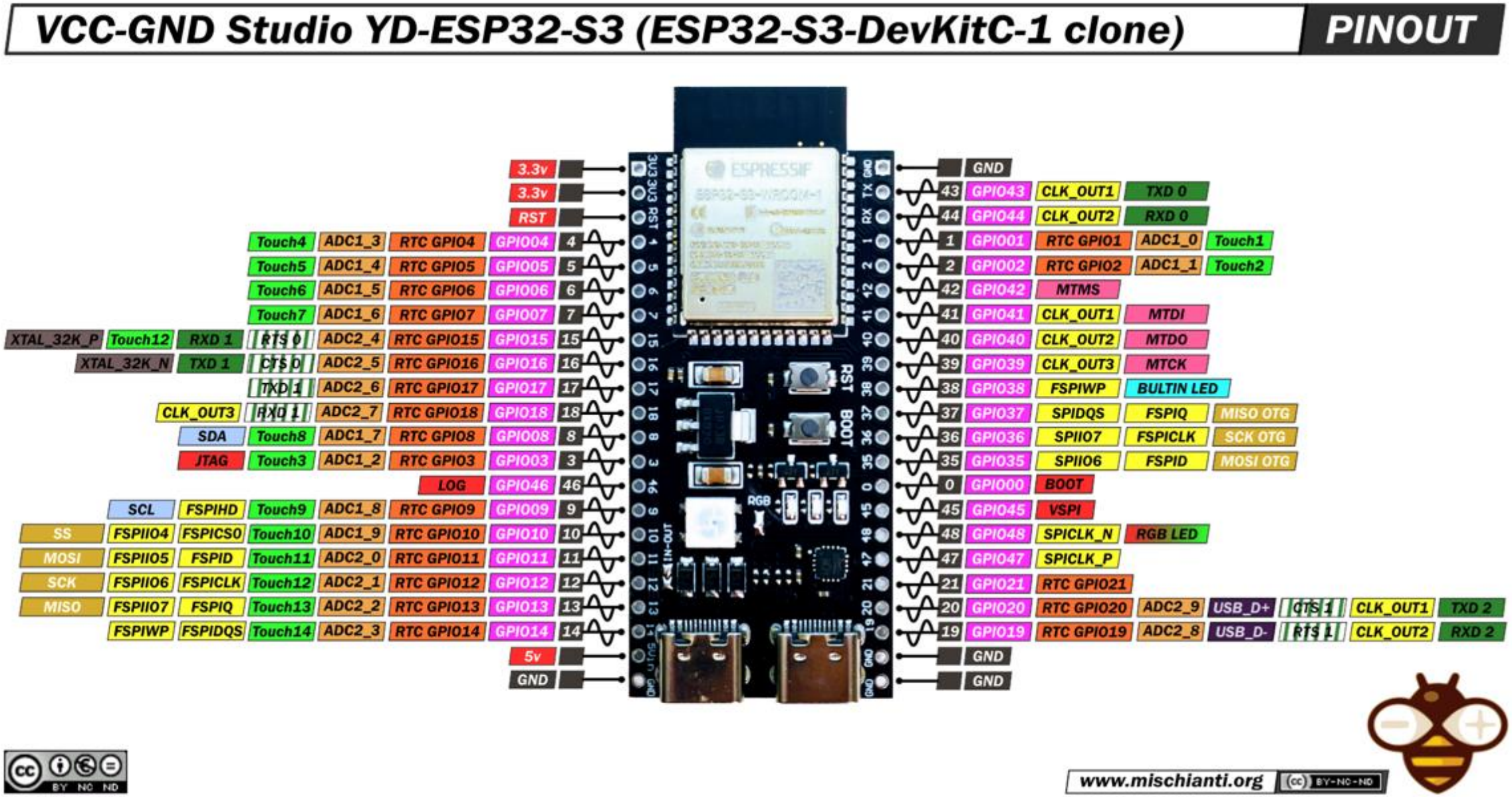
Register 6.9. GPIO_ENABLE_REG (0x0020)

GPIO_ENABLE_DATA	
31 ... 0	
0x000000	Reset

GPIO_ENABLE_DATA GPIO0~31 output enable register. (R/W)

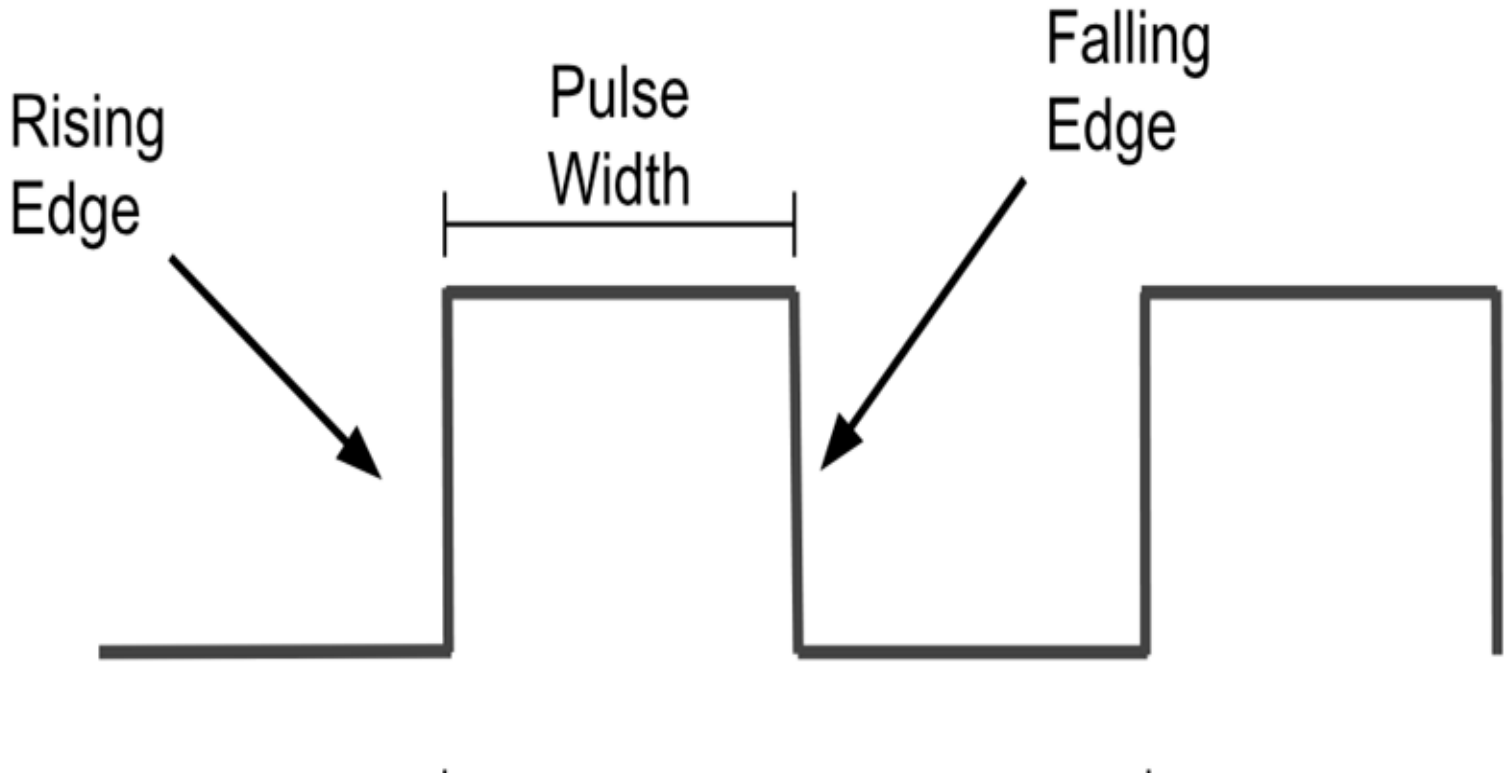


Pin Mapping


VCC-GND Studio YD-ESP32-S3 (ESP32-S3-DevKitC-1 clone)
PINOUT
3.3v
3.3v
RST
Touch4 ADC1_3 RTC GPIO4 GPIO004 4
Touch5 ADC1_4 RTC GPIO5 GPIO005 5
Touch6 ADC1_5 RTC GPIO6 GPIO006 6
Touch7 ADC1_6 RTC GPIO7 GPIO007 7
XTAL_32K_P Touch12 RXD 1 RTS 0 ADC2_4 RTC GPIO15 GPIO015 15
XTAL_32K_N TXD 1 CTS 0 ADC2_5 RTC GPIO16 GPIO016 16
TXD 1 ADC2_6 RTC GPIO17 GPIO017 17
CLK_OUT3 RXD 1 ADC2_7 RTC GPIO18 GPIO018 18
SDA Touch8 ADC1_7 RTC GPIO8 GPIO008 8
JTAG Touch3 ADC1_2 RTC GPIO3 GPIO003 3
LOG GPIO046 46
SCL FSPIHD Touch9 ADC1_8 RTC GPIO9 GPIO009 9
SS FSPII04 FSPICS0 Touch10 ADC1_9 RTC GPIO10 GPIO010 10
MOSI FSPII05 FSPID Touch11 ADC2_0 RTC GPIO11 GPIO011 11
SCK FSPII06 FSPICLK Touch12 ADC2_1 RTC GPIO12 GPIO012 12
MISO FSPII07 FSPIQ Touch13 ADC2_2 RTC GPIO13 GPIO013 13
FSPIWP FSPIDQS Touch14 ADC2_3 RTC GPIO14 GPIO014 14
5v
GND
GND
GND
43 GPIO43 CLK_OUT1 TXD 0
44 GPIO44 CLK_OUT2 RXD 0
1 GPIO01 RTC GPIO1 ADC1_0 Touch1
2 GPIO02 RTC GPIO2 ADC1_1 Touch2
42 GPIO42 MTMS
41 GPIO41 CLK_OUT1 MTDI
40 GPIO40 CLK_OUT2 MTDO
39 GPIO39 CLK_OUT3 MTCK
38 GPIO38 FSPIWP BULTIN LED
37 GPIO37 SPIDQS FSPIQ MISO OTG
36 GPIO36 SPII07 FSPICLK SCK OTG
35 GPIO35 SPII06 FSPID MOSI OTG
0 GPIO00 BOOT
45 GPIO45 VSPI
48 GPIO48 SPICLK_N RGB LED
47 GPIO47 SPICLK_P
21 GPIO21 RTC GPIO21
20 GPIO20 RTC GPIO20 ADC2_9 USB_D+ CTS 1 CLK_OUT1 TXD 2
19 GPIO19 RTC GPIO19 ADC2_8 USB_D- RTS 1 CLK_OUT2 RXD 2
GND
GND
www.mischianti.org
BY-NC-ND

Part II: Clocks on an MCU

- A clock is the "heartbeat" of a microcontroller:
  - Syncs operations
  - Coordinates tasks
- The clock produces a periodic signal (typically a square wave)



Part II: Timers on the ESP32-S3

- Timers count clock cycles or pulses
  - Used for creating delay
  - Measuring time lengths
  - Performing tasks at regular intervals
- Three main types of timers
  - General Purpose: used for a wide variety of tasks
  - Watchdog timers: used for system reliability
  - RTC: keeps track of actual date and time
- For this lab we will use the general purpose timers
  - Four 54-bit general purpose timers divided into two timer groups (Group 0 and Group 1) with two timers each.
    - 16-bit prescaler
    - Auto reload capability
    - Up/down functionality
    - Programmable alarm

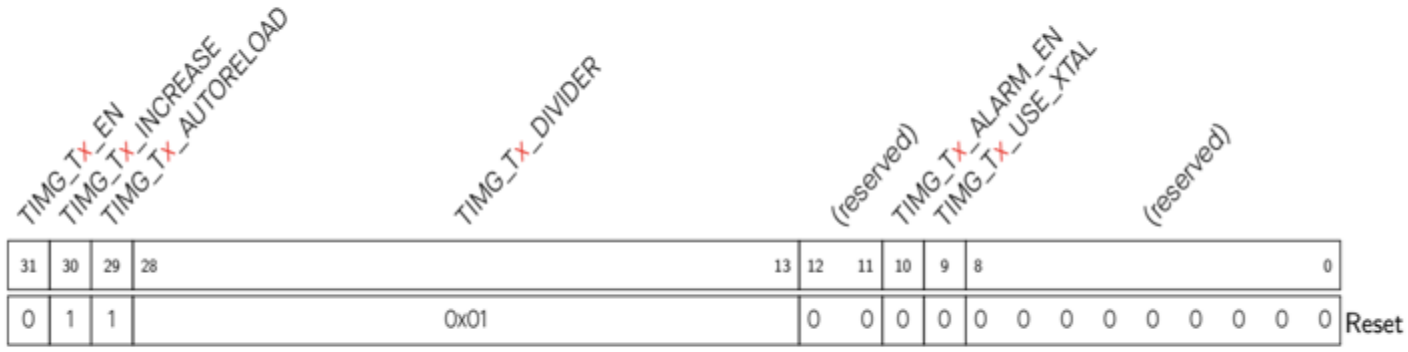
Part II: Timer Registers

- *TIMG_T0CONFIG_REG* is the configuration register
  - Enable timer and set prescaler
  - The **TIMG_T0CONFIG_REG(0)** macro stores the address of this register

- *TIMG_T0LO_REG* is the lower 32-bits of timer 0.
  - **TIMG_T0LO_REG(0)**

- *TIMG_T0UPDATE_REG* copies the current value of Timer0 to HI/LO
  - **TIMG_T0UPDATE_REG(0)**

Register 12.1. TIMG_TxCONFIG_REG (x: 0-1) (0x0000+0x24*x)



TIMG_Tx_USE_XTAL 0: Use APB_CLK as the source clock of timer group; 1: Use XTAL_CLK as the source clock of timer group. (R/W)

TIMG_Tx_ALARM_EN When set, the alarm is enabled. This bit is automatically cleared once an alarm occurs. (R/W/SC)

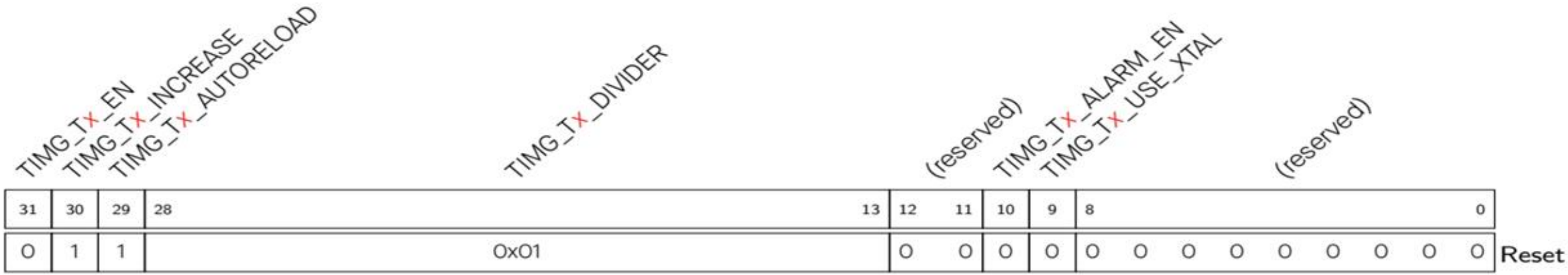
TIMG_Tx_DIVIDER Timer x clock (Tx_clk) prescaler value. (R/W)

TIMG_Tx_AUTORELOAD When set, timer x auto-reload at alarm is enabled. (R/W)

TIMG_Tx_INCREASE When set, the timer x time-base counter will increment every clock tick. When cleared, the timer x time-base counter will decrement. (R/W)

TIMG_Tx_EN When set, the timer x time-base counter is enabled. (R/W)

Register 12.1. TIMG_TxCONFIG_REG (x: 0-1) (0x0000+0x24*x)



TIMG_Tx_USE_XTAL 0: Use APB_CLK as the source clock of timer group; 1: Use XTAL_CLK as the source clock of timer group. (R/W)

TIMG_Tx_ALARM_EN When set, the alarm is enabled. This bit is automatically cleared once an alarm occurs. (R/W/SC)

TIMG_Tx_DIVIDER Timer *x* clock (Tx_clk) prescaler value. (R/W)

TIMG_Tx_AUTORELOAD When set, timer *x* auto-reload at alarm is enabled. (R/W)

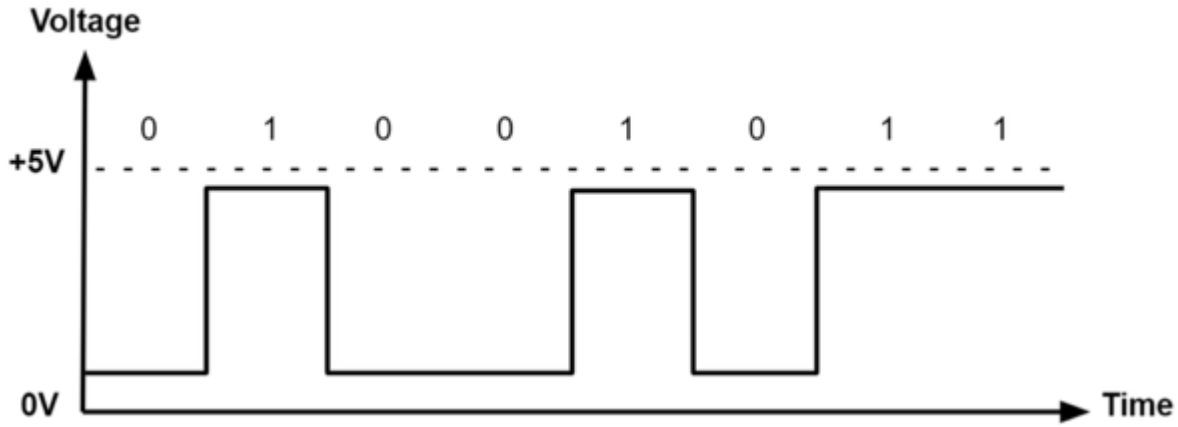
TIMG_Tx_INCREASE When set, the timer *x* time-base counter will increment every clock tick. When cleared, the timer *x* time-base counter will decrement. (R/W)

TIMG_Tx_EN When set, the timer *x* time-base counter is enabled. (R/W)

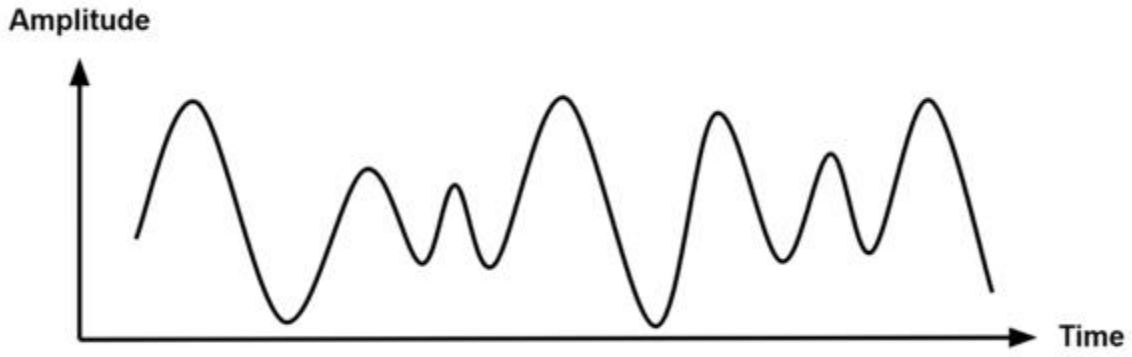
Analog I/O

- Digital signal has two states: ON and OFF

- Different peripherals need a range of values to function: motors, LEDs, buzzers, etc.



Digital Signal

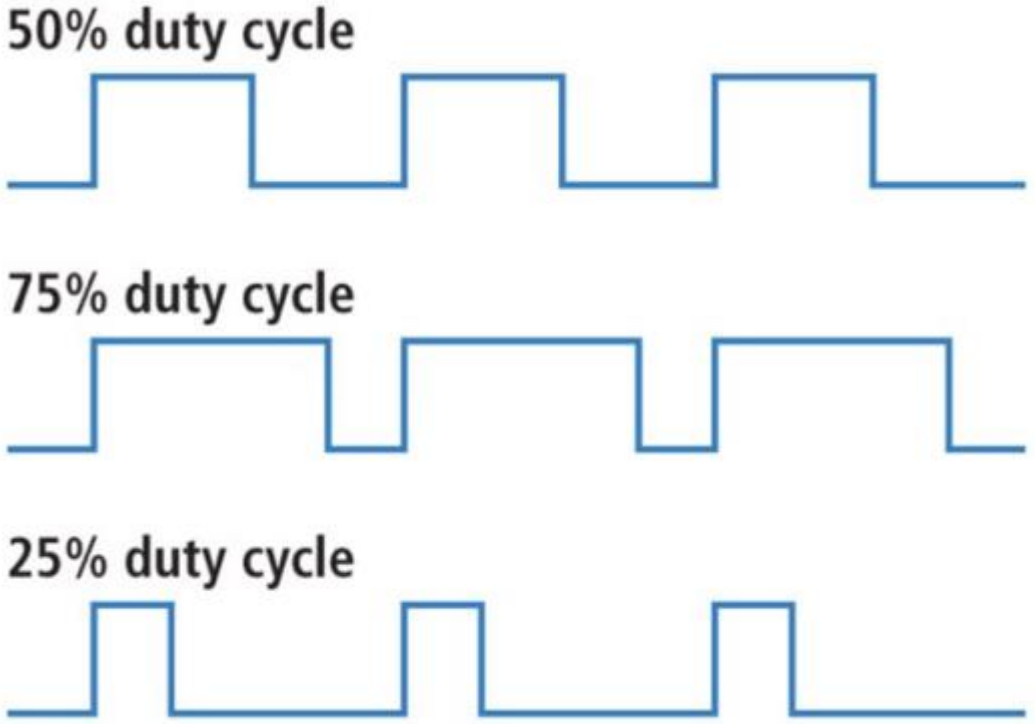


Analog Signal

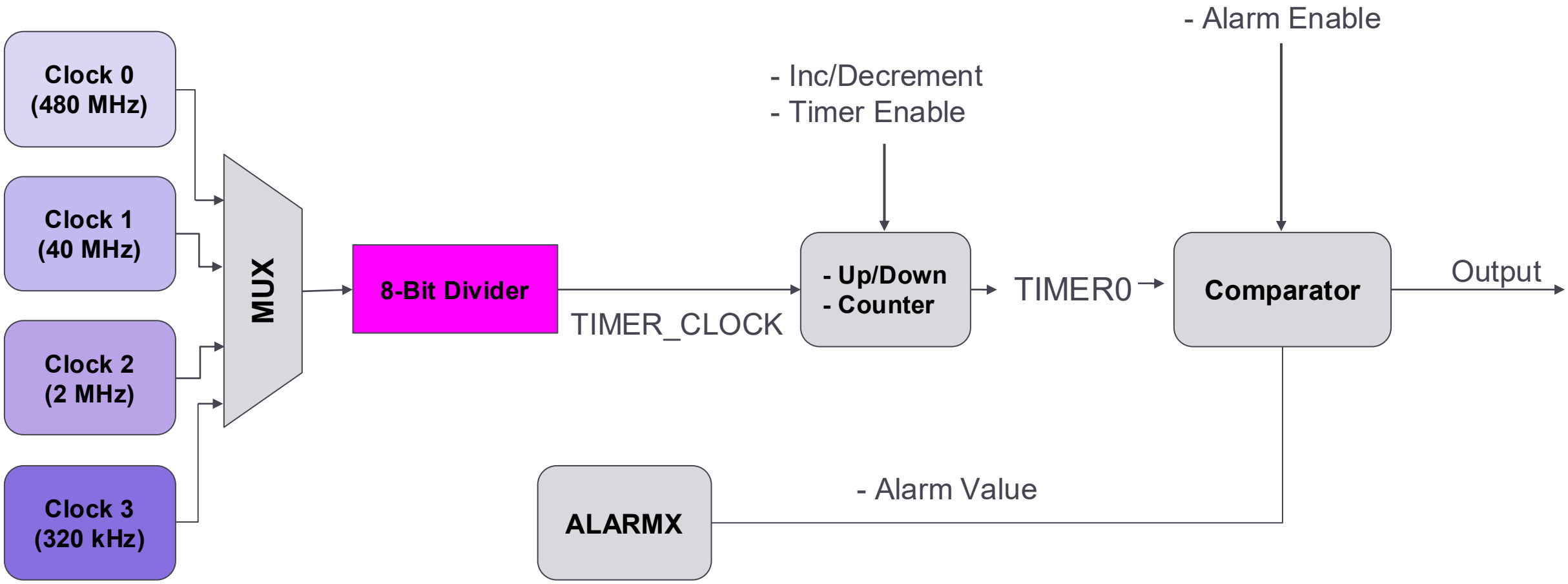
Pulse Width Modulation (PWM)

- PWM simulates this needed analog signal by switching a digital signal on and off at a high frequency.

- Key Terms:
  - *Duty Cycle*: % of time the signal is HIGH compared to the total cycle time.
  - *Frequency*: cycles/second.
  - *Resolution*: # of bits used to define the duty cycle.



https://learn.sparkfun.com/tutorials/pulse-width-modulation/all


Clock 0
(480 MHz)
Clock 1
(40 MHz)
Clock 2
(2 MHz)
Clock 3
(320 kHz)
MUX
8-Bit Divider
TIMER_CLOCK
- Inc/Decrement
- Timer Enable
- Up/Down
- Counter
TIMER0
- Alarm Enable
Comparator
Output
ALARMX
- Alarm Value

Lab Exercise

I- Given our constraints, describe the clock source and divider necessary to achieve a TIMER_CLOCK with a frequency of:

1. 60 MHz
2. 20 MHz
3. 2 MHz
4. 20 kHz
5. 2 kHz

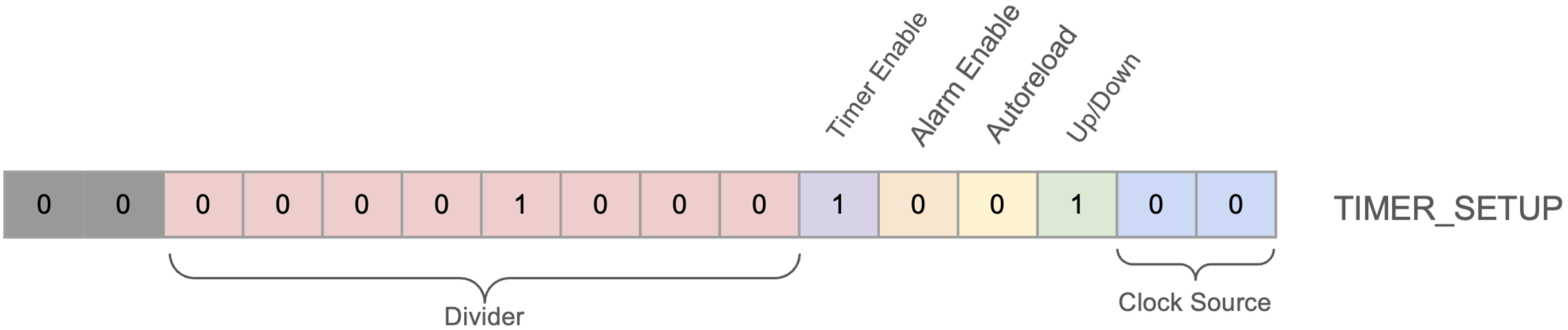
If more than one combination works, list them all.

II- Set the TIMER_SETUP and ALARM0 registers to generate:

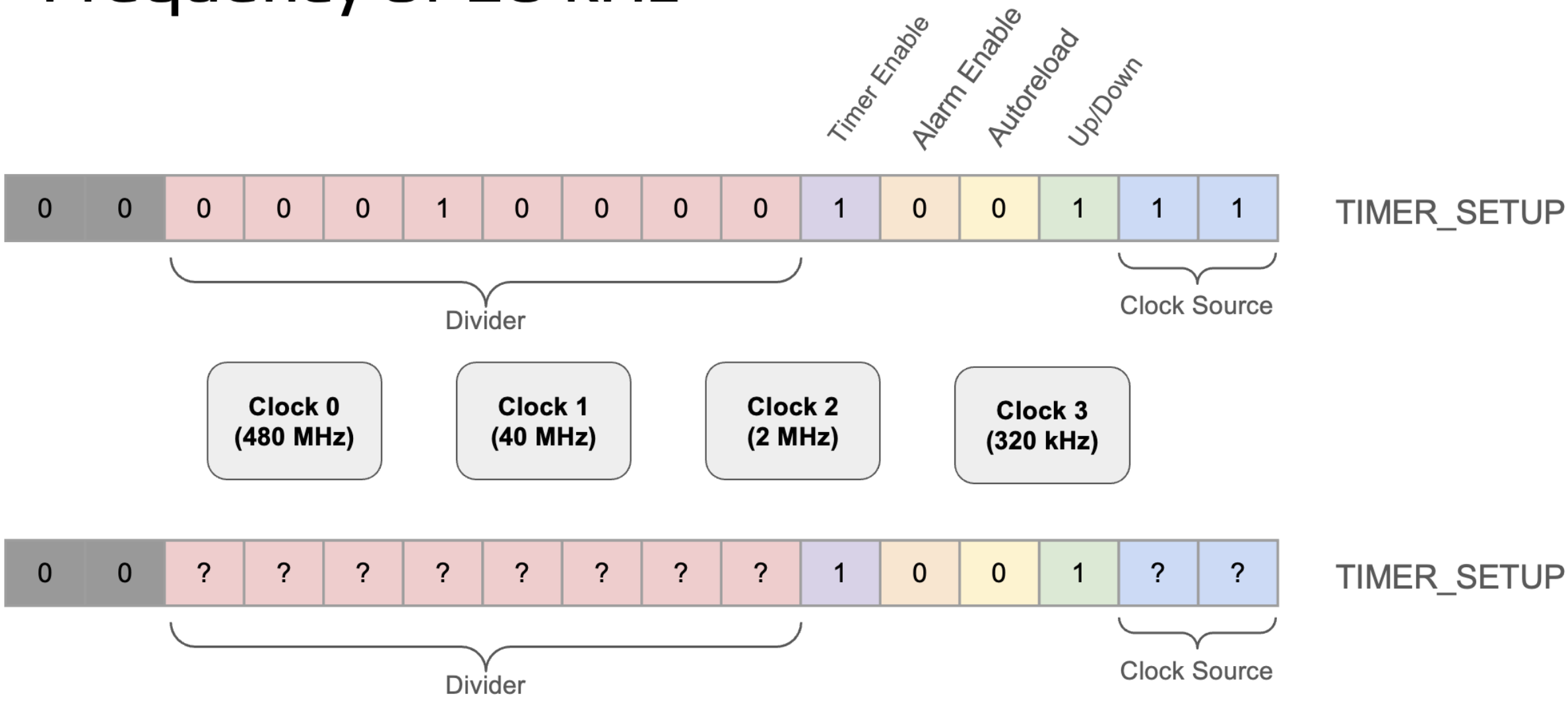
1. A one-shot (not-repeating) alarm at 20s.
2. An alarm triggered every 96 nanoseconds.
3. An output signal of 1 kHz.

Select the appropriate clock source, divider, and flags.

Frequency of 60 MHz



Frequency of 20 kHz



GPIO Control vs Registers

Objective:
Replicate controlling an external LED using direct hardware register manipulation instead of using pinMode() or digitalWrite().

Tips:

Build a circuit like in Fig a

Use volatile keyword for register variables to ensure direct memory access.

It might skip necessary updates if not used.

Don't use pinMode() or digitalWrite()

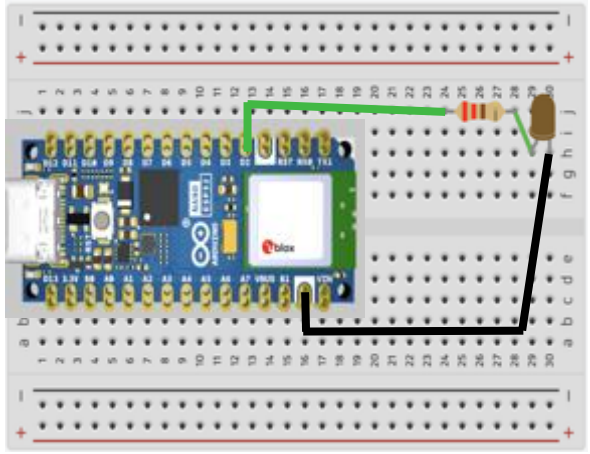


Fig a

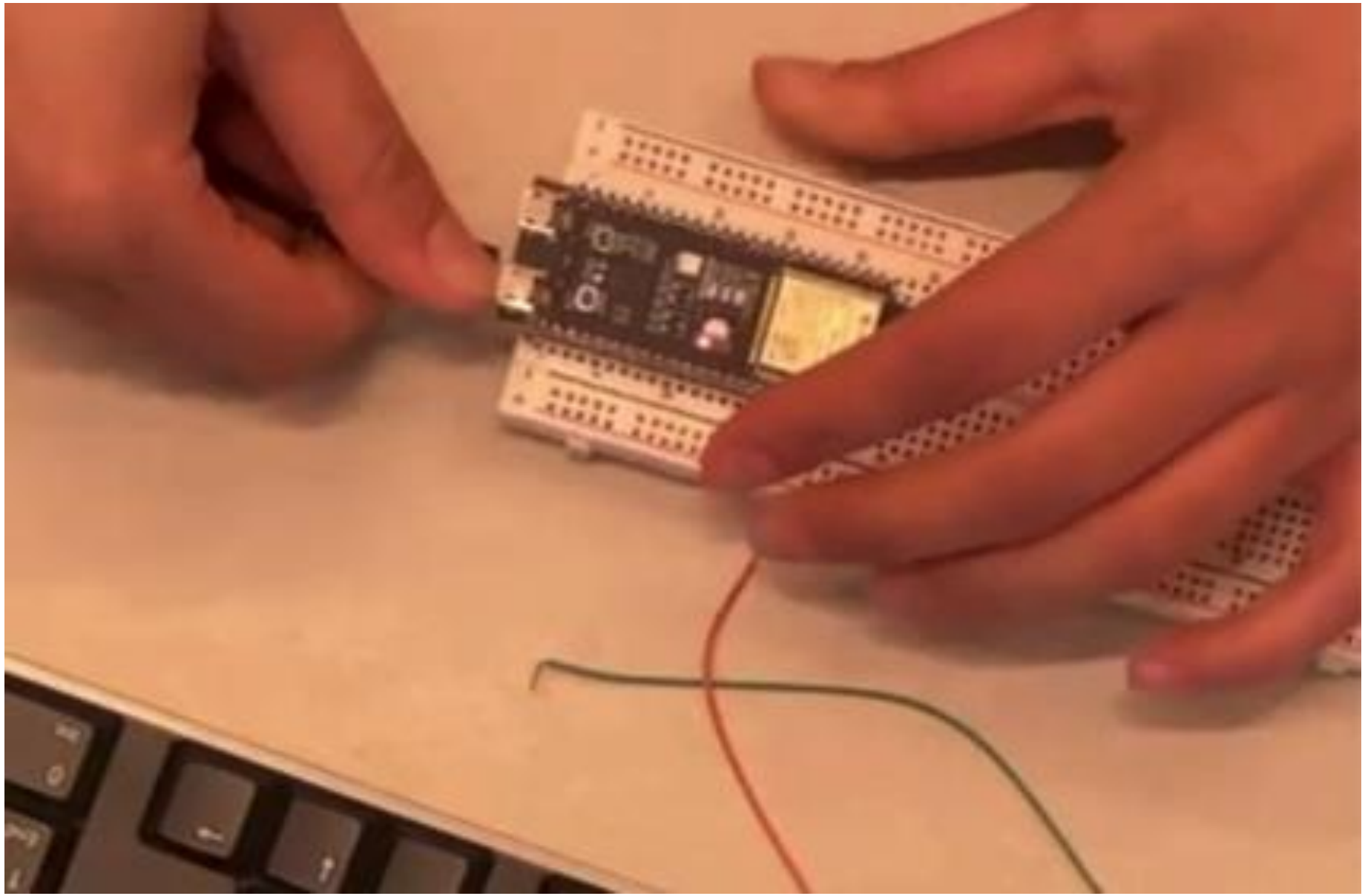
Tasks:

1. Build a circuit
2. Write your sketch to control the external LED, with the LED blinking on and off in one second intervals.

Deliverables:

1. External Led blinking Demo
2. Code

Demo Expectations



Comparing Timers

Objective:

Measure and compare the speed of using Arduino's digitalWrite() function versus direct hardware register access for toggling a GPIO pin.

Task:

1. Write Two Sketches:
    - One using digitalWrite().
    - One using direct register access.
2. Measure Execution Time:
    - Toggle GPIO pin HIGH → LOW several times.
    - Record total execution time.
3. Compare Results:
    1. Print total time for each method to Serial Monitor.

Deliverables:

Discuss in the report your comparison between Library Functions and Direct Register Access

Sketch code (with comments)

- Measuring Direct Register Access
- Measuring Arduino Library Functions

Timing & Synchronization

Objective:

Learn how to use ESP32's hardware timers to toggle an LED and measure timing with high precision, bypassing blocking functions like delay().

Task:

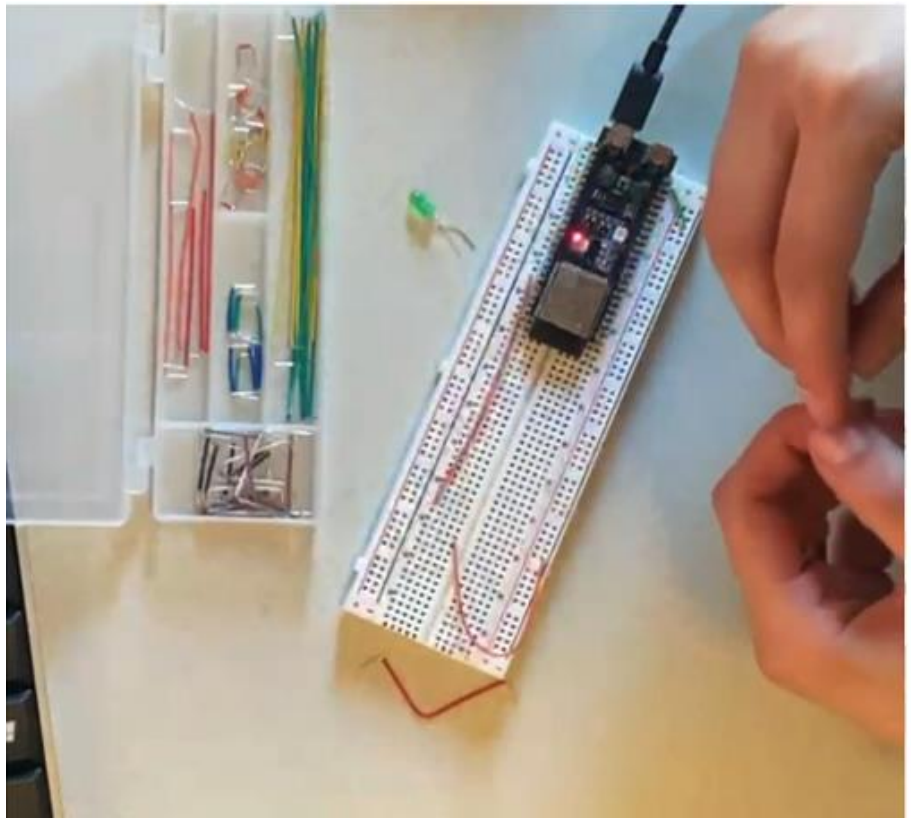
1. Reproduce LED Toggle: Replicate the functionality of a blinking LED using direct timer register access instead of delay() or library functions.
2. Measure Timing: Use an oscilloscope to analyze the LED's signal pattern and confirm correct timer functionality

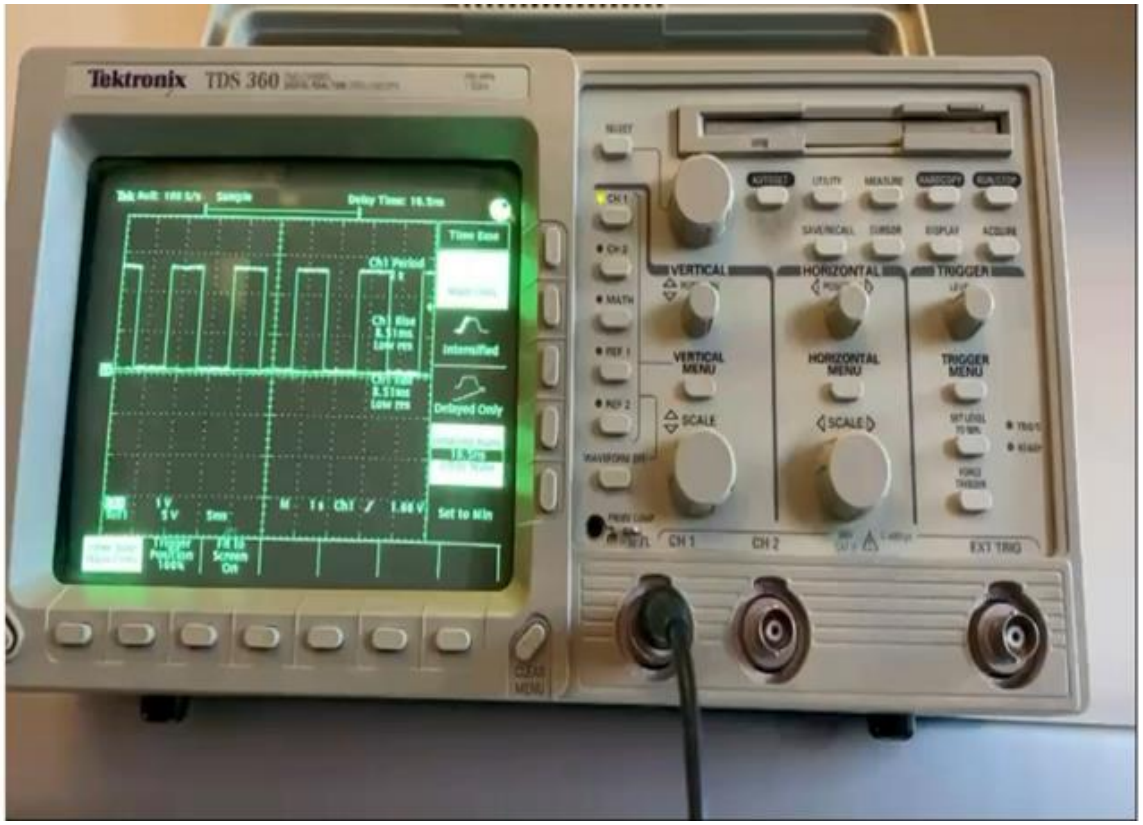
Deliverables:

A live demo of the LED blinking from with live oscilloscope measurements.

Coding sketch code (with comments)

Demo Expectations





Peripheral Interaction

Objective: Learn to control the brightness of an LED using a photoresistor, analog-to-digital conversion (ADC), and pulse-width modulation (PWM) on the ESP32.

Background:

- Photoresistors:
  - Resistance decreases with increased light intensity.
  - Useful for sensing light levels in applications like automatic lighting and alarm systems.
- Analog-to-Digital Converters (ADCs):
  - Converts analog signals into digital values for microcontroller processing.
- Pulse Width Modulation (PWM):
  - Simulates varying voltage levels by switching a digital signal on/off rapidly.
    - Duty Cycle: Percentage of time signal is HIGH.
    - Frequency: Cycles per second (reduces flickering for LEDs).
    - Resolution: Bits used to define duty cycle (higher resolution = finer control).
- LEDC Library Functions (IT IS UPDATED, YOU SHOULD FIND THE NEW VERSION):
  - ledcSetup(channel, freq, res): Configures a PWM channel.
  - ledcAttachPin(pin, channel): Assigns a pin to a PWM channel.
  - ledcWrite(channel, duty): Sets the duty cycle for the PWM channel.

Task:

1. Build the Fig b
2. Write a program to:
    1. Read analog values from the photoresistor using analogRead().
    2. Use the LEDC library to adjust LED brightness based on light intensity.
3. Demonstrate real-time brightness adjustment.

Deliverables:

A live demo of the LED responding to changes in ambient lighting.

Step 5 sketch code (with comments)

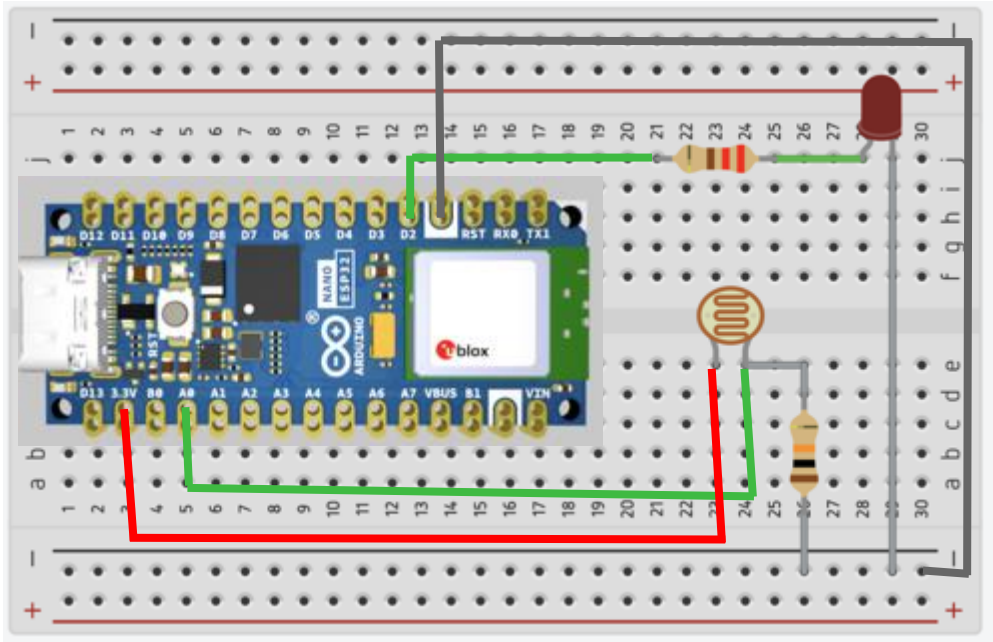


Fig b

Demo Expectations

Bonus

Objective:

Learn to produce a sequence of frequencies with a LED Light (low brightness, high brightness, higher brightness...) or by reading different frequencies on the oscilloscope based on ambient lighting conditions using ESP32. The frequencies change once when the lighting threshold is met.

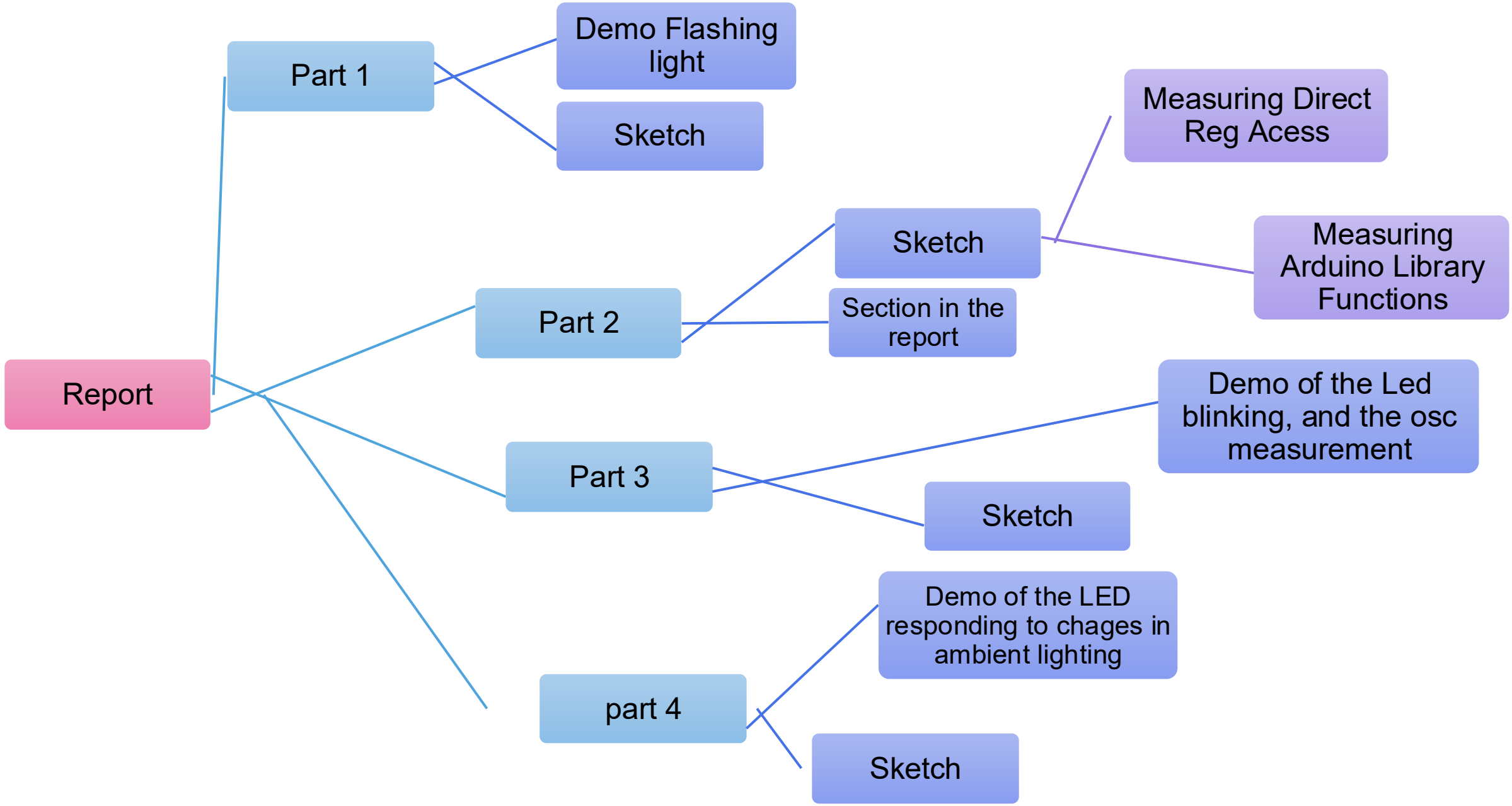
Task:

1. Build a circuit (tips in the manual).
2. Write a sketch to:
    1. Use analogRead() to monitor ambient light levels.
    2. Trigger a sequence of frequencies on the LED or the oscilloscope when a light threshold is exceeded.
    3. Stop the sequence if the threshold is no longer met.
    4. Use a timer to control the duration of each note.

Deliverables:

Sketch & Live Demo

Deliverables Recap (+POP Quiz on your Last Demo)



Office Hours